The .NET Architecture

Notes and case study: CampusPass, a digital queue, smart batching and QR gatepass system for Chuka University

COSC 434 · Group 5 · Group members: _____

1. What .NET is

.NET is Microsoft's free, open-source platform for building and running applications (web, desktop, mobile, cloud, games and IoT) in languages such as **C#**, **F#** and **Visual Basic**. Its central architectural idea is **managed execution**. A program is not compiled straight to machine code for one processor. It is compiled to a processor-independent **intermediate language**, which a runtime, the **Common Language Runtime (CLR)**, loads, verifies, compiles and supervises while it runs. That one design decision gives .NET language interoperability, memory safety and portability across Windows, Linux and macOS.

Generation	Years	Key point
.NET Framework 1.0 – 4.8.1	2002–2022	The original platform. Windows-only. Still maintained for legacy apps (Web Forms, WCF, WinForms).
.NET Core 1.0 – 3.1	2016–2019	Rewritten from scratch: open-source, cross-platform, modular, high-performance.
.NET 5 → .NET 10	2020–2025	One unified platform. A new version every November; even-numbered versions are Long-Term Support (LTS, 3 years).

CampusPass targets **.NET 8 (LTS)**, supported until **November 2026**. Moving to **.NET 10 (LTS)** only needs `TargetFramework` changed in each project file.

2. The architecture at a glance

.NET is a layered platform. Each layer depends only on the layer beneath it:

Layer	What it contains	In CampusPass
Languages & compilers	C#, F#, VB. The **Roslyn** compiler turns source code into CIL.	All code is C# 12
Application models	ASP.NET Core (web & APIs), MAUI / WPF / WinForms (desktop & mobile), worker services	ASP.NET Core **MVC** web app
Libraries (BCL + NuGet)	Collections, LINQ, I/O, cryptography, threading, time zones; NuGet packages such as EF Core	**EF Core**, `HMACSHA256`, `Rfc2898DeriveBytes`, `SemaphoreSlim`, `TimeZoneInfo`
Common Language Runtime	Assembly loader, JIT compiler, garbage collector, type safety, exceptions, threads	Runs `CampusPass.*.dll`
OS & hardware	Windows, Linux, macOS · x64, ARM64	Built on Windows, can be hosted on Linux or in Docker

3. Core components

3.1 Common Language Runtime (CLR): the execution engine

- **Class loader**: finds and loads assemblies and their dependencies, and verifies that the code is type-safe.
- **JIT compiler**: converts CIL to native machine code for the actual CPU (section 3.4).
- **Garbage collector**: allocates and frees memory automatically (section 3.5).
- **Exception handling**, the **thread pool** and **security checks**, shared by every .NET language.

3.2 Common Type System (CTS) and Common Language Specification (CLS)

The **CTS** defines every kind of type the runtime understands: **value types** (`int`, `struct`, `enum`, stored inline) and **reference types** (`class`, `interface`, `delegate`, `string`, stored on the managed heap). Every .NET language compiles to the same types, so a class written in C# can be used from F# or VB without wrappers. The **CLS** is the subset of CTS rules a library must follow to be usable from all languages; for example, a public API should not expose unsigned integers, which some languages lack.

3.3 CIL, metadata and assemblies

Compilers emit **Common Intermediate Language (CIL)** plus **metadata**, a full description of every type, method and attribute. Both are packaged into an **assembly** (`.dll` or `.exe`) whose **manifest** records its name, version and dependencies. Metadata makes **reflection** possible, and ASP.NET Core relies on it: MVC discovers `StaffController.CallNext` and binds form fields to parameters by reading metadata, and EF Core maps the `Ticket` class to a database table the same way. CIL, metadata and the runtime rules form an open standard: **ECMA-335, the Common Language Infrastructure (CLI)**.

3.4 Just-In-Time (JIT) compilation

A method is compiled to native code the first time it is called, then cached. **Tiered compilation** first produces quick, lightly optimised code (tier 0) so the app starts fast, then recompiles frequently used ("hot") methods with full optimisation (tier 1). There are two alternatives: **ReadyToRun** pre-compiles assemblies to shorten start-up, and **Native AOT** compiles the whole app ahead of time into one native executable with no JIT.

3.5 Garbage collector (GC)

The GC is **generational**. New objects start in **Gen 0**, objects that survive move to **Gen 1** and then **Gen 2**, and objects of 85,000 bytes or more go to the **Large Object Heap**. Most objects die young, so collecting Gen 0 often is cheap. The GC compacts the heap and frees unreachable objects, which removes whole classes of C/C++ bugs: memory leaks from a forgotten `free()`, dangling pointers and double frees. Web servers use **Server GC** (one heap per CPU core) for throughput.

3.6 SDK, CLI and NuGet

The **.NET SDK** provides the `dotnet` command-line tool (`new`, `build`, `run`, `test`, `publish`). **NuGet** is the package manager. CampusPass uses only two runtime packages: **Entity Framework Core** (database access) and **QRCoder** (draws the gatepass QR code). Everything else comes from the BCL and ASP.NET Core.

4. How a .NET program runs

```
C# source (.cs)  ──Roslyn──►  CIL + metadata (.dll)  ──CLR loads & JIT──►  native x64/ARM64 code  ──►  OS / CPU
```

1. We write C# source, e.g. `QueueService.cs`, which holds the smart-batching rules.
2. `dotnet build`: Roslyn compiles each project to CIL, producing `CampusPass.Domain.dll`, `CampusPass.Application.dll`, `CampusPass.Infrastructure.dll` and `CampusPass.Web.dll`.
3. `dotnet CampusPass.Web.dll`: the host starts the CLR, which loads our four assemblies, ASP.NET Core and EF Core. At start-up, EF Core **migrations** create or upgrade the SQLite database.
4. Each method is verified and JIT-compiled on its first call. Every HTTP request runs on a **thread-pool** thread, and `async`/`await` frees the thread while the database works.
5. While the app runs, the CLR manages memory (GC), exceptions and threads. `dotnet test` runs the **33 xUnit tests** against a real in-memory SQLite database.
6. `dotnet publish` (or the Dockerfile) produces a folder or container that runs on any OS with the .NET runtime.

5. ASP.NET Core: the web application model

- **Kestrel**: the built-in, cross-platform web server. It usually runs behind a proxy (IIS, Nginx or a Cloudflare tunnel); CampusPass reads the proxy's `X-Forwarded-*` headers to get the real client IP and HTTPS.
- **Generic Host**: handles start-up, **configuration** (`appsettings.json` + environment variables), logging and **dependency injection (DI)**.
- **Middleware pipeline**: every request passes through an ordered chain of components; each one can answer the request, change it or pass it on:

```
Forwarded headers ► Security headers (CSP, X-Frame-Options) ► Static files ► Routing ► Rate limiter
   ► Authentication (cookie) ► Authorization (Student / Staff / Admin) ► MVC filters ► Controller ► Razor view
```

- **MVC pattern**: **Models** (entities and page models), **Views** (Razor `.cshtml` pages mixing HTML and C#) and **Controllers** (`StudentController`, `StaffController`, `AdminController`, `VerifyController`, `AccountController`, `SetupController`).
- **Dependency injection**: services are registered once in `Program.cs` (`AddDbContext`, `AddScoped<QueueService>()`) and handed to constructors automatically, one instance per request.
- **Built-in security**: anti-forgery tokens on every form, cookie authentication, role-based `[Authorize]`, rate limiting, data protection keys, and **action filters** (e.g. forcing a password change after a one-time password).

6. Case study: CampusPass

6.1 The real-world problem

Chuka University has about **30,000 students**. Offices open Monday–Friday, 8am–5pm, with only 10–15 staff. At the start of the semester, just the continuing students take about **a month and a half** to clear. A gatepass depends on **fee clearance**, yet many students queue only to learn their balance. Those with a problem (a payment not showing, the portal failing, a missing exam card) are sent from Finance to ICT to the Registrar and back. Downloaded gatepasses are plain documents anyone can edit, so staff must check them by hand. The Finance Officer's memo gives each faculty a registration day, but students come on any day.

6.2 The solution and its functionality

Take most students out of the queue, and make the queue smart for the rest. The design borrows **HELB-style batching** (a group is given a day and time) and **bank-style ticketing** (a number, then "Call next").

Function	Who	What happens
Instant clearance & QR gatepass	Student	Home shows the fee balance. Cleared students get an **HMAC-signed QR gatepass** automatically; students who owe see exactly what to pay and where.
Request a service	Student	Pick one of 6 services (ICT & Portal, Finance, GatePass/Security, Academic, Administration, Other), then **Join Queue** (earliest batch with room) or **Book Appointment** (pick a time window). Repeat taps return the same ticket.
Smart batching	System	Each day is split into windows ("Batch 3 · 10:00–11:00"). Window capacity = staff on duty × window length ÷ the service's average minutes. Tickets are never booked before the student's **faculty registration day**.
Track & get notified	Student	Ticket shows the batch, position (3/12), estimated wait, and a live queue (numbers only). Notifications say *almost your turn* and *it's your turn*, with the office and counter. "I'm on my way" or cancel.
Call Next	Staff	Dashboard with today's queue. **Call Next** notifies the student; the same screen shows their **fee position**. Start → Complete (with quick replies) → call the next, or transfer to the right office (same ticket number).
Gate check	Guard	Any phone camera scans the QR. The public Verify page checks signature, expiry and **live** fee status, then shows VALID or NOT VALID.
Administration	Admin	Dashboard (students, departments, staff on duty, requests per service, queue health), services, users (one-time passwords), fee uploads, batches & faculty days, reports.

6.3 How the .NET architecture is applied

The solution is four assemblies plus a test project. Dependencies point **inward** (clean architecture), so the business rules never depend on the web or the database engine:

```
CampusPass.Web (ASP.NET Core MVC) ──► CampusPass.Application (rules + IAppDb interface) ──► CampusPass.Domain (entities)
CampusPass.Infrastructure (EF Core AppDbContext + SQLite migrations) ── implements IAppDb
```

.NET feature	Used in CampusPass for
EF Core + LINQ + migrations	Tickets, students, staff and settings in SQLite; the queries are written in C#; the schema is versioned. Switching to SQL Server is a provider change.
`async`/`await`, `SemaphoreSlim`	Non-blocking database calls; two officers can never call the same student, and two students can't take the last place in a batch
BCL cryptography	`HMACSHA256` signs gatepass tokens; PBKDF2 (`Rfc2898DeriveBytes`, 210,000 iterations) hashes passwords; `FixedTimeEquals` prevents timing attacks
ASP.NET Core auth, filters, rate limiter	Role-based screens, lockout after 5 failed logins, forced password change, CSRF protection
`TimeZoneInfo`	Batches use Africa/Nairobi time wherever the server runs
xUnit + `dotnet test`	33 automated tests: batching, capacity, faculty days, call order, concurrency, notifications, accounts, gatepass forgery, fee uploads

Shippable, not a demo: the code contains no test data. A fresh install prints a one-time **setup code** in the server log; the person holding it creates the first administrator. Students get logins from Finance's fee file, staff from the admin, and every issued password must be changed at first sign-in.

7. Demonstration: how it happens

1. **Cleared student** signs in → Home says *Fees cleared · Gatepass ready* → My Gatepass shows the QR.
2. **Guard** scans the QR with a phone camera → green **VALID** with name and reg no. An edited link shows **NOT VALID**.
3. **Student with a problem** → Request Service → ICT & Portal → Join Queue → ticket **ICT-001**, Batch 2 · 9:00–10:00, position 1/3, ~8 min.
4. **ICT officer** → Call Next → the student's phone shows *It's your turn · ICT Centre, Counter 1*; the next students get *You're almost up*. Start Service → Complete with a quick reply.
5. **Wrong office**: Finance transfers a ticket to Academic; the student keeps the same number.
6. **Admin** uploads new fee balances → a student who paid is told *You're cleared* and gets a gatepass; one who now owes has their QR switch to NOT VALID.

8. Strengths and limitations of .NET

Strengths	Limitations

Open-source and cross-platform (Windows, Linux, macOS, containers)	Needs the .NET runtime on the server, unless published self-contained or as Native AOT
High performance: tiered JIT, modern GC, the fast Kestrel server	JIT adds start-up time (reduced by ReadyToRun / Native AOT)
Language interoperability through the CTS/CLS	Some desktop frameworks (WPF, WinForms) and the old .NET Framework are Windows-only
Rich ecosystem: EF Core, built-in security, DI, testing	Serverless front-end hosts such as Vercel can't run it; it needs a container, VM or App Service
Strong tooling (Visual Studio, VS Code, `dotnet` CLI) and predictable LTS releases	LTS lasts 3 years, so apps must be upgraded regularly

9. .NET compared with the other architectures

Architecture	Core idea	.NET equivalent / difference
JavaBeans	Reusable Java components following naming conventions (getters/setters, events)	Properties and events are built into C# and the CTS rather than conventions
EJB	Server-side components managed by an application server (transactions, security, pooling)	ASP.NET Core + DI + EF Core provide the same services in a lightweight host with no separate app server
CORBA	Language-neutral distributed objects described in IDL, connected by an ORB	Language neutrality is achieved *inside* one runtime (CTS/CIL); remote calls use HTTP/REST or gRPC
OSGi	Java modules (bundles) installed and updated at runtime	Assemblies + `AssemblyLoadContext` for loading; NuGet for packaging
Web services	Services exposed over HTTP with SOAP/WSDL or REST	ASP.NET Core builds REST APIs and gRPC services; WCF covered SOAP in .NET Framework

10. Conclusion and future work

The managed architecture of .NET (CIL, the CTS, the CLR's JIT and GC, and a large class library), together with ASP.NET Core's middleware, MVC, dependency injection and EF Core, let a small team build a secure, testable and portable system quickly. CampusPass uses these layers to replace a physical queue with instant clearance checks, signed gatepasses, batched time windows and a bank-style call system. Next steps: **SMS** alerts (Africa's Talking), live updates with **SignalR**, a direct feed from Finance's system, SQL Server for multi-server hosting, and an upgrade to **.NET 10 LTS**.

References: Microsoft Learn, *.NET documentation*, *ASP.NET Core fundamentals* and *Entity Framework Core* (learn.microsoft.com) · ECMA-335, *Common Language Infrastructure* · .NET support policy (dotnet.microsoft.com/platform/support/policy) · Chuka University Finance Office, registration memo CU/FIN/MEMO/VOL.1/81 (28 Sep 2026).